Software Engineering Lab/ 3th Grade

[Second Semester] and [2026]

[Lab 3] [17/2/ 2026] [8:30 + 10:30]

Instructor Information

Instructor	Email	Hours
Lec. Najwan Abed	[najwan.abedhassan@nahrainuniv.edu.iq]	[2 Hrs]
Assist. Lect. Abbass	[abbas.kh.abbas@nahrainuniv.edu.iq]	
Assist. Prof. Zahraa Jaaz	[zahraa.jaaz@nahrainuniv.edu.iq]	

Flow Control

- ✓ Using Boolean logic
- ✓ Branching code
- ✓ Looping code

Boolean Logic
The Foundation

Boolean logic forms the basis of all decision-making in code. The bool type holds only two values: true or false.
Named after mathematician George Boole, this system enables programs to make intelligent decisionsbased on conditions.

BooleanComparison Operators

OPERATOR	CATEGORY	EXAMPLE EXPRESSION	RESULT
`==`	Binary	`var1 = var2 == var3;`	`var1` is assigned the value `true` if `var2` is equal to `var3`, or `false` otherwise.
`!=`	Binary	`var1 = var2 != var3;`	`var1` is assigned the value `true` if `var2` is not equal to `var3`, or `false` otherwise.
`<`	Binary	`var1 = var2 < var3;`	`var1` is assigned the value `true` if `var2` is less than `var3`, or `false` otherwise.
`>`	Binary	`var1 = var2 > var3;`	`var1` is assigned the value `true` if `var2` is greater than `var3`, or `false` otherwise.
`<=`	Binary	`var1 = var2 <= var3;`	`var1` is assigned the value `true` if `var2` is less than or equal to `var3`, or `false` otherwise.
`>=`	Binary	`var1 = var2 >= var3;`	`var1` is assigned the value `true` if `var2` is greater than or equal to `var3`, or `false` otherwise.

Instructor Information

Instructor	Email	Hours
Lec. Najwan Abed	[najwan.abedhassan@nahrainuniv.edu.iq]	[2 Hrs]
Assist. Lect. Abbass	[abbas.kh.abbas@nahrainuniv.edu.iq]	
Assist. Prof. Zahraa Jaaz	[zahraa.jaaz@nahrainuniv.edu.iq]	

Conditional Boolean Operators

OPERATOR	CATEGORY	EXAMPLE EXPRESSION	RESULT
&&	Binary	`var1 = var2 && var3;`	`var1` is assigned the value `true` if `var2` and `var3` are both `true`, or `false` otherwise. (Logical AND)
||	Binary	`var1 = var2 || var3;`	`var1` is assigned the value `true` if either `var2` or `var3` (or both) is `true`, or `false` otherwise. (Logical OR)

Performance Advantage

Conditional operators improve performance by skipping unnecessary evaluations Use && and || instead of & and | for better efficiency

1- Highest Priority: Prefix operators: ++, --, unary +, -, !, ~

2- Arithmetic Operations: Multiplication, division, modulus, then addition and subtraction

3- Comparison Operators: Relational operators: <, >, <=, >=, ==, !=

4- Logical Operations: Bitwise, then conditional: &, ^, |, &&, ||

5-Assignment Operators: All assignment operations including compound assignments

Software Engineering Lab/ 3th Grade

[Second Semester] and [2026]

[Lab 3] [17/2/ 2026] [8:30 + 10:30]

Instructor Information

Instructor	Email	Hours
Lec. Najwan Abed	[najwan.abedhassan@nahrainuniv.edu.iq]	[2 Hrs]
Assist. Lect. Abbass	[abbas.kh.abbas@nahrainuniv.edu.iq]	
Assist. Prof. Zahraa Jaaz	[zahraa.jaaz@nahrainuniv.edu.iq]	

Operator Precedence

PRECEDENCE	OPERATORS
Highest	++, -- (used as prefixes); (), +, - (unary), !, ~
	*, /, %
	+, -
	<<, >>
	<, >, <=, >=
	==, !=
	&
	^
	|
	&&
	||
	=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, |=
Lowest	++, -- (used as suffixes)

Boolean Logic & Clarity

Use Parentheses for Clarity

Compiler knows operator order, but humans forget. Make expressions explicit:

var1 = (var2 <= 4) && (var2 >= 2);

Parentheses solve ambiguity and allow easy reordering

BRANCHING

Three Branching Techniques

Ternary Operator

Simplest comparison for quick conditional assignments

if Statement

Versatile decision-making for conditional execution

switch Statement

Multiple value testing in single structure

Instructor Information

Instructor	Email	Hours
Lec. Najwan Abed	[najwan.abedhassan@nahrainuniv.edu.iq]	[2 Hrs]
Assist. Lect. Abbass	[abbas.kh.abbas@nahrainuniv.edu.iq]	
Assist. Prof. Zahraa Jaaz	[zahraa.jaaz@nahrainuniv.edu.iq]	

1- Ternary Operator

Syntax

```
<test> ? <resultIfTrue> : <resultIfFalse>
```

Works on three operands, returns one of two values

Example

```
string resultString = (myInteger < 10)
  ? "Less than 10"
  : "Greater than or equal to 10";
```

Evaluates test, assigns appropriate string based on result

2-The if Statement

1- Simple if Execute code only if test is true

2- if-else Execute different code for true vs. false

3- Code blocks Use braces for multiple lines in each branch

Instructor Information

Instructor	Email	Hours
Lec. Najwan Abed	[najwan.abedhassan@nahrainuniv.edu.iq]	[2 Hrs]
Assist. Lect. Abbass	[abbas.kh.abbas@nahrainuniv.edu.iq]	
Assist. Prof. Zahraa Jaaz	[zahraa.jaaz@nahrainuniv.edu.iq]	

if Statement Example

Simple if:

```
if (<test>)
    <code executed if <test> is true>;
```

if-else

```
if (<test>)
    <code executed if <test> is true>;
else
    <code executed if <test> is false>;
```

If-else Block

```
if (<test>)
{
  <code executed if <test> is true>;
}
else
{
  <code executed if <test> is false>;
}
```

Nested if approach

```
if (var1 == 1)
{
  // Do something.
}
else
    {
        if (var1 == 2)
          {
            // Do something else.
          }
          else
              {
                if (var1 == 3 || var1 == 4)
                  {
                    // Do something else.
                  }
                  else
                      {
                        // Do something else.
                      }
              }
    }
```

Instructor Information

Instructor	Email	Hours
Lec. Najwan Abed	[najwan.abedhassan@nahrainuniv.edu.iq]	[2 Hrs]
Assist. Lect. Abbass	[abbas.kh.abbas@nahrainuniv.edu.iq]	
Assist. Prof. Zahraa Jaaz	[zahraa.jaaz@nahrainuniv.edu.iq]	

2- The switch Statement

Test multiple discrete values efficiently

Key Features

- Compare test variable to multiple values
- Each case requires break, return, or goto
- Stack cases for multiple conditions
- Default handles unmatched values

C# Difference

Unlike C++, flow cannot continue from one case to next—must explicitly break or redirect

Basic Structure of switch statement

```
switch (<testVar>)
        {
                case <comparison_val_1>:
                                < code to execute if <testVar> == <comparison_val_1> >
                                break;
                case <comparison_val_2>:
                                < code to execute if <testVar> == <comparison_val_2> >
                                break;
                …
                …
                case <comparison_val_n>:
                                < code to execute if <testVar> == <comparison_val_n> >
                                break;
                default:
                        < code to execute if <testVar> != <comparison_vals> >
                        break;
        }
```